Implementing JWT Authentication in a Customer Portal API



Estimated time needed: 45 minutes

In this lab, we will explore the process of integrating JSON Web Token (JWT) authentication into a Node.js based API designed for a Customer Portal. JSON Web Tokens serve as a widely accepted method for securely exchanging information in JSON format between different entities. They play a vital role in user authentication and securing routes within web applications, ensuring enhanced security and reliability.

Learning Objective:

After completing this lab, you will be able to:

- Set up a basic Node.js server using Express.js to create API endpoints.
- Implement JWT authentication for user login, secure API routes for authenticated access, and use sample data to generate JWT tokens for protected route access.
- Understand the workflow of JWT authentication by testing the endpoints using cURL.

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands on labs for course and project related labs. Theia is an open source IDE (Integrated Development Environment).

Important Notice about this lab environment

Please be aware that sessions for this lab environment are not persisted. Every time you connect to this lab, a new environment is created for you. Any data you may have saved in the earlier session would get lost. Plan to complete these labs in a single session, to avoid losing your data.

The content of this lab is licensed under Apache 2.0

Task 1: Setup: Creating Directory & Initializing Node.js

1. Open a terminal window by using the menu in the editor: Terminal > New Terminal.

2. Change to your project folder, if you are not in the project folder already.

```
cd /home/project
```
Run

3. Create a directory for your Customer Portal project using below command

```
mkdir jwt-customer-portal
```
Run

This command creates a new directory named "jwt-customer-portal ".

4. Navigate into the newly created directory using the command.

```
cd jwt-customer-portal
```
Run

This command changes the current directory to "jwt-customer-portal".

5. Initialize a new Node.js project within this directory. Run the command:

```
npm init -y
```
Run

This command initializes a new Node.js project, automatically accepting default configurations using the -y flag, eliminating the need for manual prompts.

6. Install the necessary dependencies by executing:

```
npm install express jsonwebtoken
```
Run

Install essential packages (express framework and jsonwebtoken) using npm for your node-jwt-api application.

The content of this lab is licensed under Apache 2.0

Task 2: Create API Server

1. Create an index.js file using below command.

```
touch index.js
```

Run

2. Paste the below code in index.js.

```
// index.js

const express = require('express');
const jwt = require('jsonwebtoken');

const app = express();
const secretKey = 'yourSecretKey'; // Replace with your own secret key

app.use(express.json());

// Sample user data (Replace with your database or actual authentication logic)
const users = [];

// Endpoint for user registration
app.post('/register', (req, res) => {
  const { username, password } = req.body;

  // Check if the username already exists
  const existingUser = users.find((u) => u.username === username);
  if (existingUser) {
    return res.status(400).json({ message: 'Username already exists' });
  }

  // Add new user to the database
  const newUser = {
    id: users.length + 1,
    username,
    password,
  };
  users.push(newUser);

  res.status(201).json({ message: 'User registered successfully' });
});

// Endpoint for user login
app.post('/login', (req, res) => {
  const { username, password } = req.body;

  // Find user by username and password
  const user = users.find((u) => u.username === username && u.password === password);

  if (user) {
    // User authenticated, generate token
    const token = jwt.sign({ id: user.id, username: user.username }, secretKey);
    res.json({ token });
  } else {
    res.status(401).json({ message: 'Invalid credentials' });
```

The content of this lab is licensed under Apache 2.0

```
// Protected route example (Dashboard access)
app.get('/dashboard', verifyToken, (req, res) => {
  // Return dashboard data or user-specific information
  res.json({ message: 'Welcome to the Customer Portal!' });
});

// Middleware to verify token
function verifyToken(req, res, next) {
  const token = req.headers['authorization'];

  if (typeof token !== 'undefined') {
    jwt.verify(token, secretKey, (err, authData) => {
      if (err) {
        res.sendStatus(403);
      } else {
        req.authData = authData;
        next();
      }
    });
  } else {
    res.sendStatus(401);
  }
}

// Start server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

1. **Server Setup:** Utilizes Express.js to create the API server.
2. **User Registration:** Provides a /register endpoint for unique username and password sign-up. Checks and adds new user data to the users array upon successful registration.
3. **User Login:** Offers a /login endpoint to verify user credentials against stored data. Generates a JWT token for successful login authentication.
4. **Protected Route:** Defines a /dashboard endpoint accessible only with a valid JWT token. Displays a welcome message upon successful access.
5. **Token Verification Middleware:** Contains middleware (verifyToken) to authenticate tokens in the Authorization header. Grants access to valid tokens; otherwise, denies access.
6. **Server Start:** Listens on port 3000 for requests. Confirms server startup upon successful launch.
7. Run your Node.js server:

```
node index.js
```

Run

The content of this lab is licensed under Apache 2.0

Task 3: Testing the API using cURL

1. Open a terminal window for cURL:

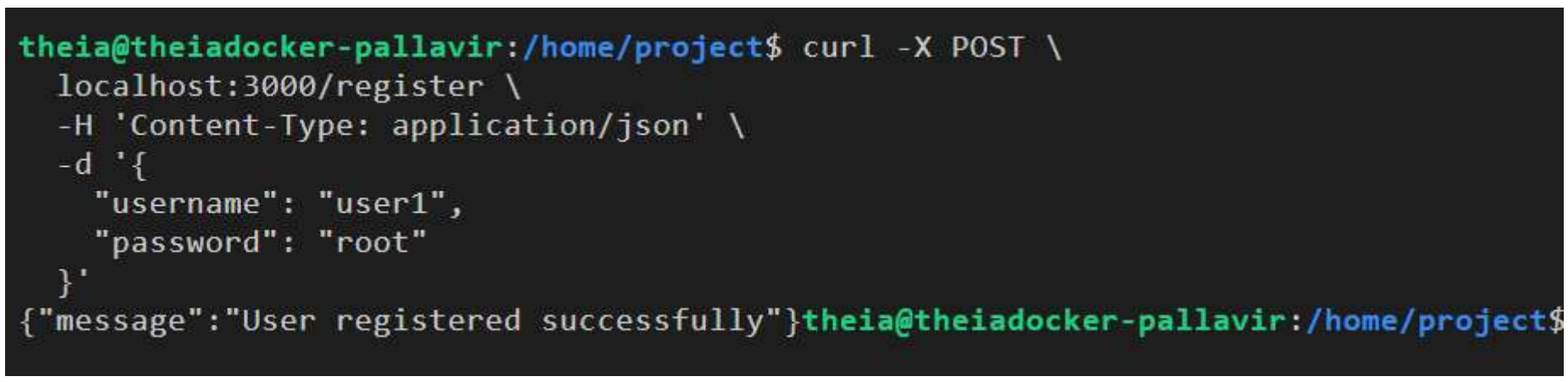
- Open a terminal window by using the menu in the editor: Terminal > New Terminal.

2. Register a user using cURL:

- Execute the below command in the terminal. This cURL command performs a POST request to register a new user.

```
curl -X POST \
  localhost:3000/register \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "example_user",
    "password": "example_password"
  }'
```

> *Note: Replace `example_user` & `example_password` with your username and password.*



3. Login a user using cURL:

```
curl -X POST \
  localhost:3000/login \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "example_user",
    "password": "example_password"
  }'
```

> *Note: Adjust the username and password values in the JSON payload to match the credentials you used for registration. Copy the token provided as output for above command.*

- This cURL command performs a POST request to login and generates a JWT token.
- Copy the token provided in the output.

The content of this lab is licensed under Apache 2.0

```
theia@theiadocker-pallavir:/home/project$ curl -X POST \
  localhost:3000/login \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "user1",
    "password": "root"
  }'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwidXNlcm5hbWUiOiJ1c2VyMSIsIm
lhdCI6MTcwNDg1NTEwOH0.nCBsj5C-lqAPIbODZwyy2300ATiCntzbX-4DiAEtW_k"}theia@theiadocker-
pallavir:/home/project$
```

4. Access the protected route using cURL:

```
curl -X GET \
  localhost:3000/dashboard \
  -H 'Authorization:  YOUR_JWT_TOKEN'
```

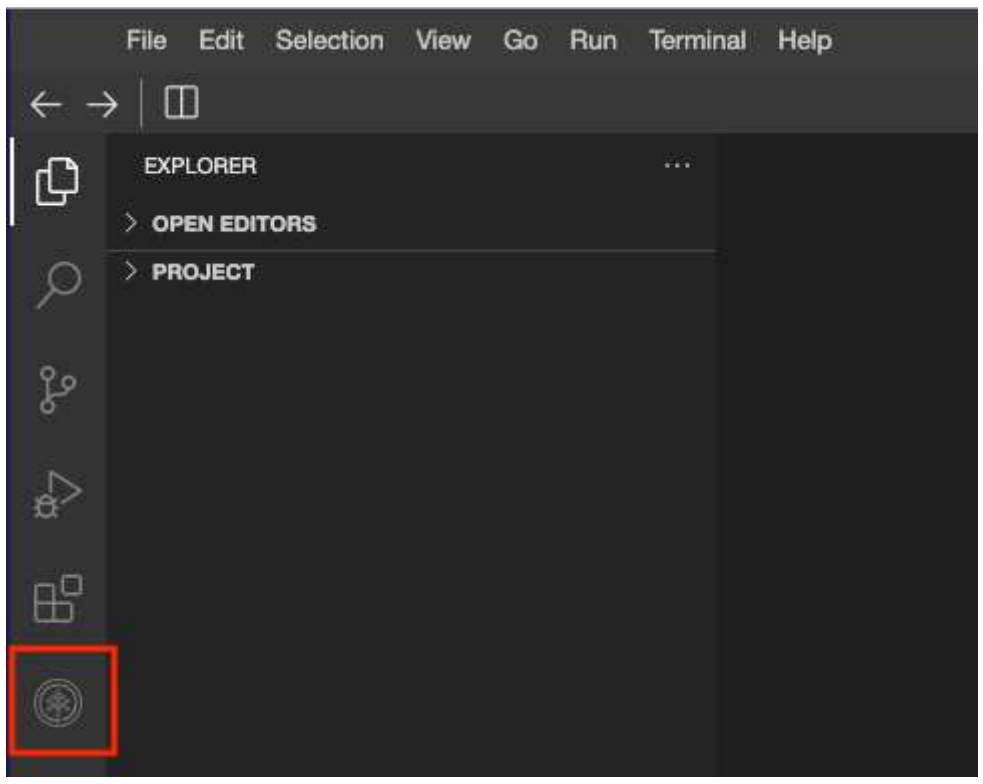
> *Note: Replace `YOUR_JWT_TOKEN` with the actual JWT token obtained after a successful login.*

```
theia@theiadocker-pallavir:/home/project$ curl -X GET \
  localhost:3000/dashboard \
  -H 'Authorization:  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwidXNlcm5hbWUiOi
J1c2VyMSIsImlhdCI6MTcwNDg1NTEwOH0.nCBsj5C-lqAPIbODZwyy2300ATiCntzbX-4DiAEtW_k'
{"message":"Welcome to the Customer Portal!"}theia@theiadocker-pallavir:/home/project
$
```

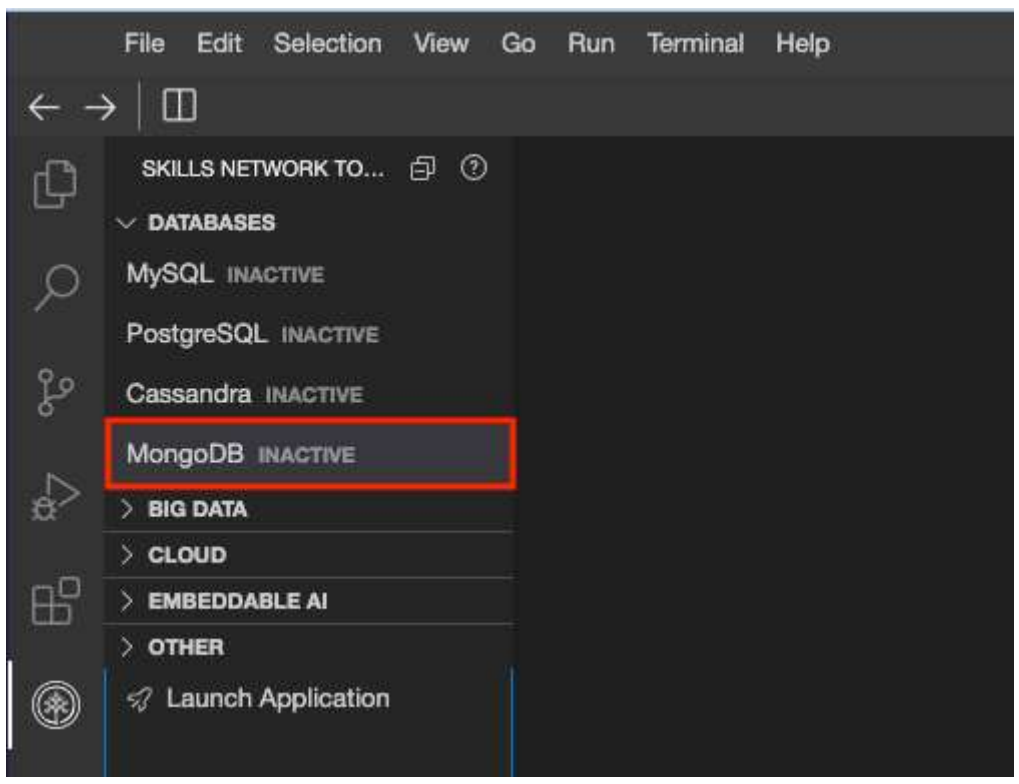
The content of this lab is licensed under Apache 2.0

Task 4: Implement JWT Authentication for User Registration Endpoint

1. Start MongoDb server

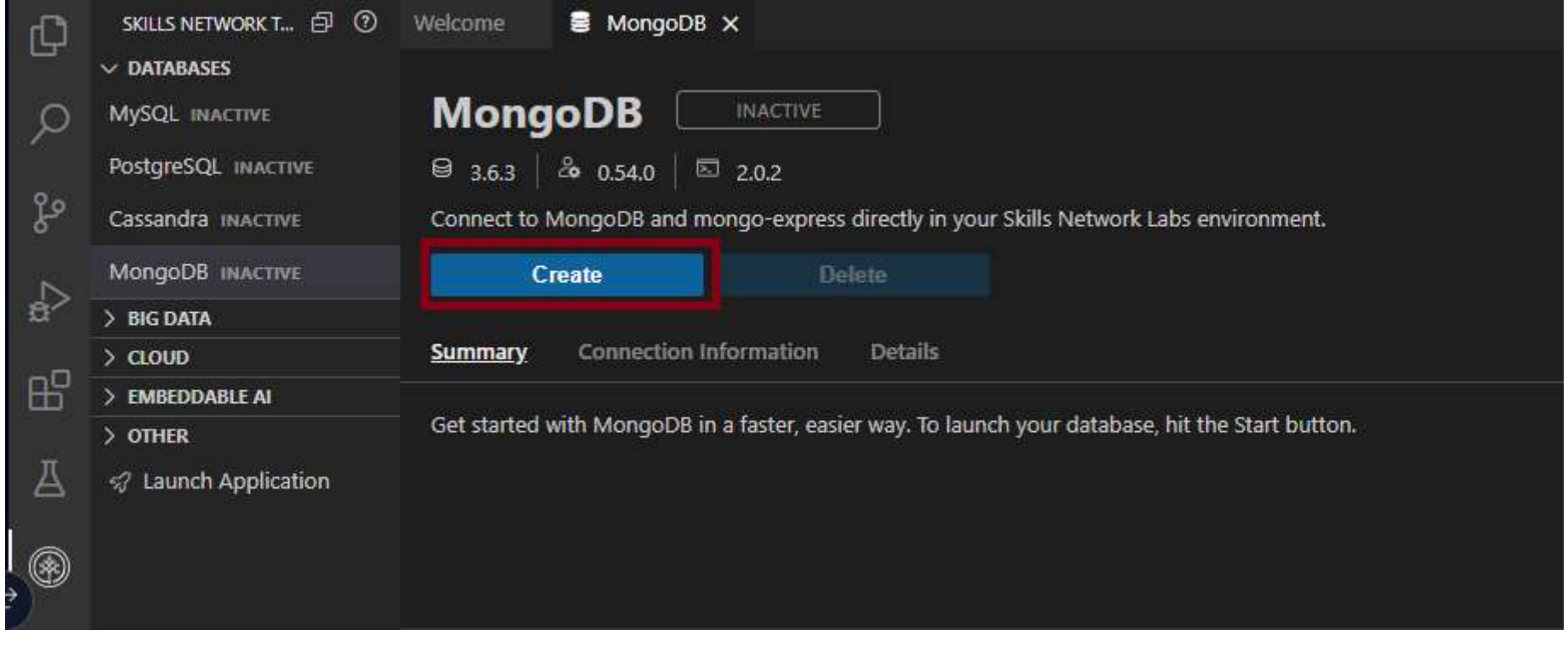


You will notice MongoDB listed there, but inactive. Which means the database is not available to use.



Once you click on it, you will see more details about it and a button to start it.

The content of this lab is licensed under Apache 2.0

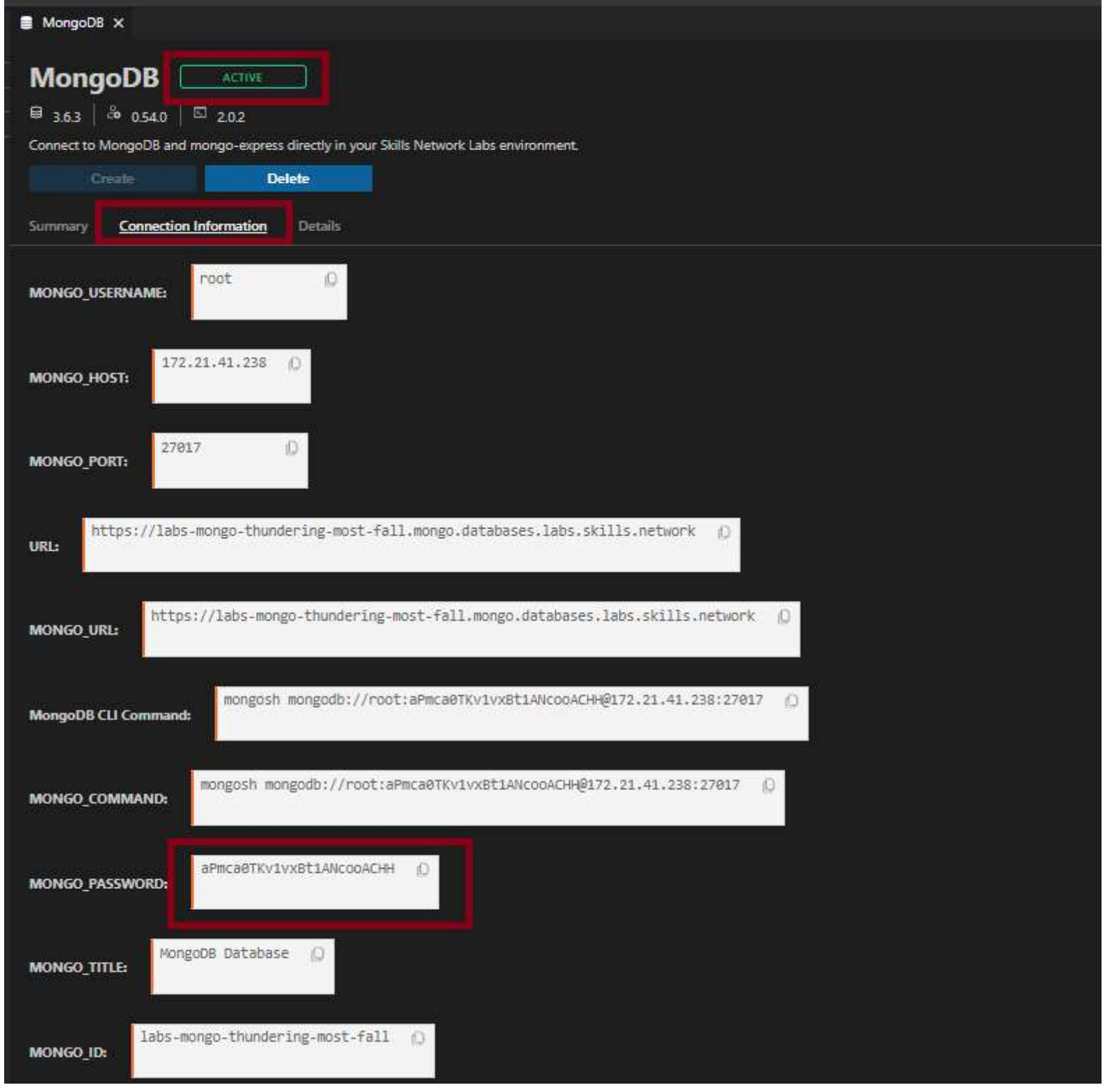


Clicking on the start button will run a background process to configure and start your MongoDB server.



Shortly after that, your server is ready for use. This deployment has access control enabled and MongoDB enforces authentication. So, **take note of the password** as you will need it to login as `root` user. `Note:` For Password and other information click on `Connection Information`

The content of this lab is licensed under Apache 2.0



2. Open a terminal window by using the menu in the editor: Terminal > New Terminal.

3. Run the below command to clone the customerportal repository

```
git clone https://github.com/ibm-developer-skills-network/abltc-backend-nodejs-customerportal.git
```

4. Change to the new directory that you just cloned.

```
cd abltc-backend-nodejs-customerportal
```

5. Run the following command to install bcrypt

```
npm install bcrypt
```

6. Install the necessary dependencies by executing:

```
npm install express jsonwebtoken
```
Run

7. Include the provided code snippet for JWT Authentication in the customer_app.js file to enable user registration with JWT Authentication.

Replace the placeholder password in Line 22 with the actual password allocated by your instance of MongoDB server.

The content of this lab is licensed under Apache 2.0

```
// Importing necessary libraries and modules
const mongoose = require('mongoose');
const Customers = require('./customer');
const express = require('express');
const bodyParser = require('body-parser');
const path = require('path');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');  // Added JWT library
const saltRounds = 5;
const secretKey = 'your-secret-key';  // Replace with a secure secret key

// A dictionary object to store username and password
let usersdic = {};

// Creating an instance of the Express application
const app = express();

// Setting the port number for the server
const port = 3000;

// MongoDB connection URI and database name
const uri = 'mongodb://root:yourpassword@localhost:27017';
mongoose.connect(uri, { dbName: 'customerDB' });

// Middleware to parse JSON requests
app.use('*', bodyParser.json());

// Serving static files from the 'frontend' directory under the '/static' route
app.use('/static', express.static(path.join('.', 'frontend')));

// Middleware to handle URL-encoded form data
app.use(bodyParser.urlencoded({ extended: true }));

// POST endpoint for user login with JWT authentication
app.post('/api/login', async (req, res) => {
    const data = req.body;
    console.log(data);

    const user_name = data['user_name'];
    const password = data['password'];

    const user = usersdic[user_name];
    if (!user || !(await bcrypt.compare(password, user.hashedpwd))) {
        res.status(401).send('User Information incorrect');
        return;
    }

    // No need to print details to the terminal, just send a success message
    res.status(200).send('Login successfully');
});

// POST endpoint for adding a new customer with JWT authentication
app.post('/api/add_customer', async (req, res) => {
    const data = req.body;
    console.log(data);

    const documents = await Customers.find({ user_name: data['user_name'] });
    if (documents.length > 0) {
        res.status(409).send('User already exists');
        return;
    }
```

The content of this lab is licensed under Apache 2.0

```
    usersdic[data['user_name']] = { hashedpwd };

    // Creating a new instance of the Customers model with data from the request
    const customer = new Customers({
        user_name: data['user_name'],
        age: data['age'],
        password: hashedpwd,
        email: data['email'],
    });

    // Saving the new customer to the MongoDB 'customers' collection
    await customer.save();

    res.status(201).send('Customer added successfully');
});

// GET endpoint for the root URL, serving the home page
app.get('/', async (req, res) => {
    res.sendFile(path.join(__dirname, 'frontend', 'home.html'));
});
```

```
// Function to authenticate JWT token
function authenticateToken(req, res, next) {
    const token = req.headers['authorization'];

    if (!token) {
        res.sendStatus(401);
        return;
    }

    jwt.verify(token, secretKey, (err, user) => {
        if (err) {
            res.sendStatus(403);
            return;
        }

        req.user = user;
        next();
    });
}

// Starting the server and listening on the specified port
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
});
```

8. Run the customer_app.js from the terminal.

```
node customer_app.js
```

Run

9. Click on the button below to launch the application.

Customer Portal

▶ Click here if the button above doesn't work

10. Verify the output:

The content of this lab is licensed under Apache 2.0



In this lab, you have used JSON Web Tokens (JWT) for user authentication. When a user logs in, a JWT containing user information is generated and sent to the client. For subsequent requests to protected routes, the server verifies the JWT using a secret key, allowing access if the verification is successful; otherwise, it sends a 403 Forbidden response.

Conclusion

Congratulations! You have successfully completed the implementation of JWT Authentication in your Customer Portal API lab.

In this lab, you learned how to:

- Successfully implemented JWT Authentication in the Customer Portal API during the lab.
- Utilized the curl command to verify authentication.
- Seamlessly incorporated JWT into the customer portal application for enhanced security and access control.

Authors

Pallavi Rai

© IBM Corporation. All rights reserved.

Task 3: Testing the API using cURL

1. Open a terminal window for cURL:

- Open a terminal window by using the menu in the editor: Terminal > New Terminal.

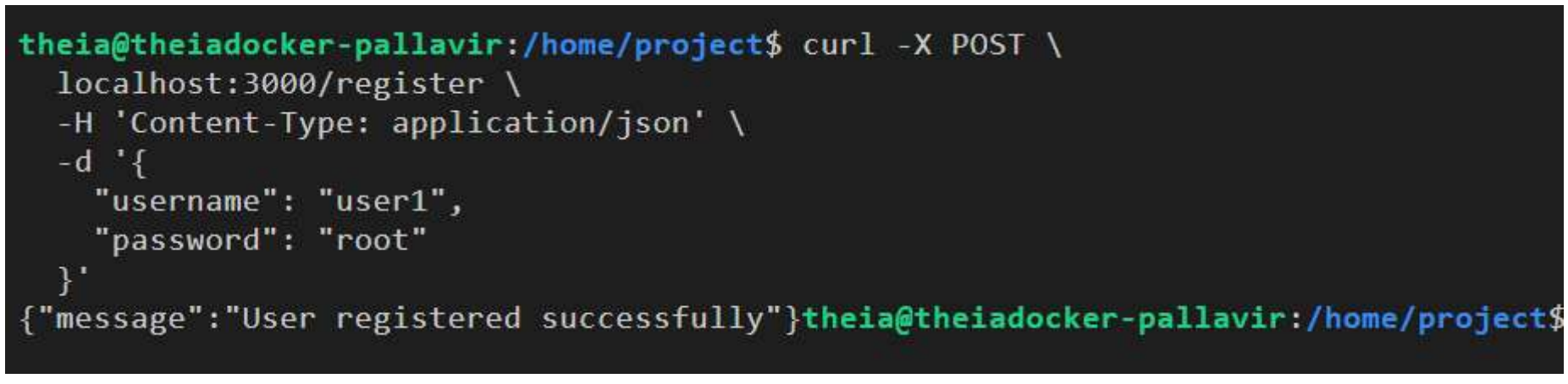
2. Register a user using cURL:

- Execute the below command in the terminal. This cURL command performs a POST request to register a new user.

The content of this lab is licensed under Apache 2.0

```
curl -X POST \
  localhost:3000/register \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "example_user",
    "password": "example_password"
  }'
```

> *Note: Replace `example_user` & `example_password` with your username and password.*



3. Login a user using cURL:

```
curl -X POST \
  localhost:3000/login \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "example_user",
    "password": "example_password"
  }'
```

> *Note: Adjust the username and password values in the JSON payload to match the credentials you used for registration. Copy the token provided as output for above command.*

- This cURL command performs a POST request to login and generates a JWT token.
- Copy the token provided in the output.



4. Access the protected route using cURL:

```
curl -X GET \
  localhost:3000/dashboard \
  -H 'Authorization:  YOUR_JWT_TOKEN'
```

The content of this lab is licensed under Apache 2.0

Note: Replace `YOUR_JWT_TOKEN` *with the actual JWT token obtained after a successful login.*

```
theia@theiadocker-pallavir:/home/project$ curl -X GET \
  localhost:3000/dashboard \
  -H 'Authorization:  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwidXNlcm5hbWUiOi
J1c2VyMSIsImlhdCI6MTcwNDg1NTEwOH0.nCBsj5C-lqAPIbODZwyy2300ATiCntzbX-4DiAEtW_k'
{"message":"Welcome to the Customer Portal!"}theia@theiadocker-pallavir:/home/project
$
```

← Task 2: Create API Server

Task 4: Implement JWT A… →

The content of this lab is licensed under Apache 2.0